P.Vishwa Nandan(192473015)

TASK B3 – STATEMENT VS PREPAREDSTATEMENT VS BATCH (8 MARKS)

1. Aim

To compare the performance and correctness of JDBC insertion methods using MySQL.

The task implements and compares:

1. Statement
2. PreparedStatement
3. PreparedStatement with addBatch() and executeBatch()

It also measures insertion time for 1,000, 10,000, and 100,000 bus-pass records and investigates the difference between auto-commit and transaction-based batch insertion.

2. Problem Description

The transport office initially stores bus-pass records in a text file. As the number of records increases, storing and managing them in a database becomes more useful.

The experiment moves bus-pass records into a MySQL table using three JDBC approaches.

The program must:

- Insert and verify 10 records using all three approaches.
- Check column names and data types using ResultSetMetaData.
- Measure insertion time for different data sizes.
- Compare auto-commit with a single transaction.
- Demonstrate what happens when an invalid record occurs halfway through insertion.

Note: The original A1 bus-pass design contains six fields, including validity date. However, the supplied B3 Java template uses five fields and does not include validity date. This implementation follows the five fields specified in the supplied B3 template.

3. MySQL Database and Table Creation

3.1 Create the Database

Run the following SQL commands in MySQL Workbench.

SQL Program – buspass_database.sql

```
CREATE DATABASE IF NOT EXISTS assignment_persistence;


USE assignment_persistence;
```

3.2 Create the Bus-Pass Table

```
CREATE TABLE IF NOT EXISTS bus_pass (

    pass_id VARCHAR(20) PRIMARY KEY,

    reg_no VARCHAR(20) NOT NULL,

    name VARCHAR(100) NOT NULL,

    route_no INT NOT NULL,

    amount DECIMAL(10,2) NOT NULL

) ENGINE = InnoDB;
```

Explanation

Column	Data Type	Description
pass_id	VARCHAR(20)	Unique bus-pass ID
reg_no	VARCHAR(20)	Student registration number
name	VARCHAR(100)	Student name
route_no	INT	Bus route number
amount	DECIMAL(10,2)	Amount paid

Why use DECIMAL for amount?

DECIMAL stores monetary values accurately to the specified number of decimal places. It is more suitable for money than FLOAT.

3.3 Verify the Table

```
DESC bus_pass;


SELECT * FROM bus_pass;
```

The table is now ready for JDBC operations.

4. Understanding the Three JDBC Approaches

4.1 Statement

Statement directly executes an SQL query created as a string.

Example:

```
Statement st = con.createStatement();


String sql =
   "INSERT INTO bus_pass VALUES " +
   "('P1001', '192311001', 'Arun', 12, 4500.00)";


st.executeUpdate(sql);
```

Explanation:

- The SQL query is created manually as a string.
- Values are joined into the query.
- The database executes the generated SQL statement.

Limitation: Joining user input directly into SQL can create SQL injection vulnerabilities.

4.2 PreparedStatement

PreparedStatement uses placeholders (?) for values.

Example:

```
String sql =
   "INSERT INTO bus_pass VALUES (?, ?, ?, ?, ?)";


PreparedStatement ps = con.prepareStatement(sql);


ps.setString(1, "P1001");
ps.setString(2, "192311001");
ps.setString(3, "Arun");
ps.setInt(4, 12);
```

ps.setDouble(5, 4500.00);

ps.executeUpdate();

Explanation:

- SQL structure is prepared separately.
- Values are supplied through parameters.
- It improves security and allows the same SQL statement to be reused.

4.3 PreparedStatement with Batch Processing

Batch processing allows multiple records to be added to a batch and executed together.

Example:

PreparedStatement ps = con.prepareStatement(sql);

ps.setString(1, "P1001");

ps.setString(2, "192311001");

ps.setString(3, "Arun");

ps.setInt(4, 12);

ps.setDouble(5, 4500.00);

ps.addBatch();

ps.executeBatch();

Explanation:

- addBatch() adds an insertion to the batch.
- executeBatch() executes the collected insertions.
- Multiple records can be sent together instead of executing each insertion separately.

This reduces the number of database communication calls.

5. B3.1 – Correctness on Small Data (2 Marks)

5.1 Aim

To insert 10 bus-pass records using Statement, PreparedStatement, and batch processing and verify that all records are stored correctly.

5.2 Java Program – PassLoader.java

This program completes the three TODO sections from the supplied question.

The method inserts records in batches of 1,000 and commits all records as one transaction.

5.3 Explanation of the TODO Sections

TODO 1 – Set Parameters and Execute Batch

The five fields are assigned using:

- setString() for pass ID.
- setString() for registration number.
- setString() for name.
- setInt() for route number.
- setDouble() for amount.

addBatch() adds each record to the batch.

Whenever 1,000 records are collected, executeBatch() sends them to the database.

Any remaining records are executed after the loop.

TODO 2 – Commit the Transaction

con.commit();

This permanently saves all successfully inserted records.

TODO 3 – Rollback on Failure

con.rollback();

If an SQL error or invalid record occurs, rollback cancels every insertion made within the current transaction.

The exception is then passed to the calling program.

5.4 Java Program – PassJdbcExperiment.java

Important: The Statement benchmark uses generated values containing only controlled letters and numbers. It is not suitable for untrusted user input. For a real application, use PreparedStatement.

5.5 ResultSetMetaData Explanation

The program uses:

ResultSetMetaData md = rs.getMetaData();

It obtains information about the selected database columns.

The following methods are used:

- getColumnCount() – Returns the number of columns.
- getColumnLabel(i) – Returns the column name or alias.
- getColumnTypeName(i) – Returns the database column type.

This verifies whether the database contains the expected fields and data types.

5.6 Expected Verification Output

The following is the expected structure of the output for each approach. The generated pass IDs and student names will be displayed by the program.

Verification using Statement

Column Information:

pass_id : VARCHAR

reg_no : VARCHAR

name : VARCHAR

route_no : INT

amount : DECIMAL

Stored Records:

P000001 | REG0000001 | Student1 | 2 | 3001.00

P000002 | REG0000002 | Student2 | 3 | 3002.00

...

Total records: 10

The same verification is performed for PreparedStatement and PreparedStatement Batch.

Expected result for all three approaches: 10 records are inserted correctly, and the five column types are verified.

6. B3.2 – Increase the Data Size (2 Marks)

6.1 Aim

To compare the insertion time of Statement, PreparedStatement, and PreparedStatement Batch for different numbers of records.

The experiment uses:

- 1,000 records.
- 10,000 records.
- 100,000 records.

6.2 Performance Measurement

The program uses:

long start = System.nanoTime();

to record the starting time.

After insertion:

long end = System.nanoTime();

The total execution time is calculated using:

double timeMs =

(end - start) / 1_000_000.0;

The table is cleared before every experiment so that earlier records do not affect the next test.

6.3 Performance Results Table

Run the program and enter the actual execution times displayed on your computer.

Number of Records	**Statement Time (ms)**	**PreparedStatement Time (ms)**	**Batch Time (ms)**
1,000	_____	_____	_____
10,000	_____	_____	_____
100,000	_____	_____	_____

Note: These values must be collected by running the Java program with your local MySQL server. Execution time depends on the computer, database configuration, JDBC driver, and system workload. Therefore, fixed timing values are not assumed here.

6.4 Expected Observation

Statement:

Each record is inserted using a newly constructed SQL string. It generally has more SQL processing and communication overhead.

PreparedStatement:

The same parameterized SQL statement is reused. It reduces repeated SQL construction and provides safer parameter handling.

PreparedStatement Batch:

Multiple records are collected and sent together. It reduces the number of database communication calls and is generally more efficient for large datasets.

As the number of records increases, the benefit of batch processing is expected to become more noticeable.

7. B3.3 – Auto-Commit vs Transaction (2 Marks)

7.1 Aim

To compare batch insertion with auto-commit enabled against batch insertion performed inside one transaction.

The experiment also introduces an invalid amount halfway through the records to observe how each approach handles failure.

7.2 Auto-Commit ON

When auto-commit is enabled:

con.setAutoCommit(true);

The database commits completed operations automatically.

In this experiment, the batch size is 1,000.

The first 1,000 records are executed successfully and committed.

The program then prepares the next records. At record 1,501, it encounters an invalid amount.

The Java amount validation throws an exception before the next batch is executed.

Therefore, the first 1,000 records remain in the table.

7.3 Single Transaction

When using a single transaction:

con.setAutoCommit(false);

All batch insertions belong to one transaction.

The program:

1. Inserts the first 1,000 records into the transaction.
2. Collects the next records.
3. Encounters the invalid amount at record 1,501.

4. Detects the error and calls rollback().
5. Removes all uncommitted insertions.

Therefore, no records remain in the table.

7.4 Forced Failure Test

The experiment creates 2,000 records and replaces the amount of record 1,501 with:

INVALID_AMOUNT

The program executes both tests:

- Batch insertion with auto-commit ON.
- Batch insertion inside a single transaction.

7.5 Expected Output

Auto-commit test: Invalid amount detected.

Rows remaining with auto-commit ON: 1000

Transaction test: Invalid amount detected.

Rows remaining after rollback: 0

This result demonstrates the difference between the two approaches.

Auto-commit ON: Previously committed records remain after the failure.

Single transaction: All insertions are rolled back when the failure occurs.

7.6 Comparison

Feature	**Auto-Commit ON**	**Single Transaction**
Commit	Automatically	Explicit commit
Rollback of all records	No	Yes
Failure halfway	Earlier committed records remain	All uncommitted records are removed
Data consistency	Partial insertion possible	All-or-nothing insertion
Suitable for	Independent small operations	Large related imports

8. B3.4 – Engineering Trade-Off (2 Marks)

8.1 Comparison of Three Approaches

Feature	Statement	PreparedStatement	PreparedStatement Batch
SQL construction	Manual string	Parameterized query	Parameterized query
Reusability	SQL string must be rebuilt	Same statement reused	Same statement reused
SQL injection protection	Unsafe when joining user input	Protects bound parameters	Protects bound parameters
Large data insertion	Usually slower	Better than repeated Statement in many cases	Generally fastest of these three
Database calls	One execution per record	One execution per record	One execution per batch
Recommended use	Simple controlled SQL	Single-record operations	Bulk insertion

8.2 Which Approach Is Best for a Nightly Import of 100,000 Passes?

For a nightly import of 100,000 bus-pass records, I would choose:

PreparedStatement with Batch Processing and a Transaction.

Reasons:

1. It inserts multiple records together.
2. It reduces database communication overhead.
3. It avoids rebuilding the SQL query for every record.
4. It uses parameterized queries.
5. A transaction ensures that either the complete import succeeds or the uncommitted changes are rolled back.
6. Batches of 1,000 records limit the number of records held in the JDBC batch at one time.

8.3 Which Approach Is Best for a Single Pass Issued at the Counter?

For issuing a single bus pass at the counter, I would choose:

PreparedStatement with a single executeUpdate().

Reasons:

1. Only one record needs to be inserted.

2. A batch is unnecessary for a single record.
3. PreparedStatement safely handles the supplied values.
4. It provides a simple and reliable insertion method.
5. Auto-commit can save the individual record immediately.

8.4 Alternative – Multi-Row INSERT

Instead of sending 1,000 individual batch entries, multiple records can be inserted using one SQL statement.

Example:

INSERT INTO bus_pass

(pass_id, reg_no, name, route_no, amount)

VALUES

('P1001', 'REG0000001', 'Arun', 12, 4500.00),

('P1002', 'REG0000002', 'Divya', 7, 3800.00),

('P1003', 'REG0000003', 'Ravi', 5, 4200.00);

Advantages

- Multiple rows are inserted using one SQL statement.
- It reduces the number of database communication calls.
- It can perform well for bulk insertion.

Limitations

- The SQL statement becomes large for many records.
- It must remain within MySQL packet and query-size limits.
- SQL values must be safely encoded or bound.
- Large statements consume additional memory while being constructed.

For large datasets, the records can be divided into smaller multi-row INSERT statements.

8.5 Alternative – MySQL LOAD DATA

MySQL also supports LOAD DATA for importing records from a file.

Example:

LOAD DATA LOCAL INFILE 'passes.csv'

INTO TABLE bus_pass

FIELDS TERMINATED BY ','

LINES TERMINATED BY '\n'

(pass_id, reg_no, name, route_no, amount);

Advantages

1. It is designed for bulk data loading.
2. It can import many records efficiently.
3. It avoids executing a separate INSERT statement for every record.
4. It is useful for large file-based imports.

Limitations

1. The file must be accessible to the client or server, depending on the command used.
2. The required MySQL permissions and local-file settings must be configured.
3. The file format and field order must match the table.
4. Invalid records still require validation and error handling.

9. Complexity Analysis

Let:

- n = Total number of bus-pass records.
- b = Number of records in one batch.

Approach	**Total Processing Time**	**Application/Batch Memory**	**Database Communication**
Statement	O(n)	O(1) additional insertion memory	Approximately n executions
PreparedStatement	O(n)	O(1) additional insertion memory	Approximately n executions
PreparedStatement Batch	O(n)	O(b) for the batch	Approximately ceil(n/b) batch executions
Multi-row INSERT	O(n)	O(b) for a chunk of b rows	Approximately ceil(n/b) executions
LOAD DATA	O(n) processing	Usually bounded streaming memory	Bulk-loading operation

Explanation

Statement:

The program constructs and executes one SQL statement for every record.

PreparedStatement:

The SQL is prepared once and reused, but each record is still executed separately.

PreparedStatement Batch:

The program collects b records before executing a batch. Therefore, the number of batch executions is approximately:

ceil(n / b)

For 100,000 records and a batch size of 1,000:

100,000 / 1,000 = 100 batches

Multi-row INSERT:

A chunk of records is included in one INSERT statement. The application must construct or bind the values for that statement.

LOAD DATA:

The database processes the input file as a bulk-loading operation. It is particularly suitable for large file imports.

The actual execution time depends on database configuration, network communication, indexing, storage speed, and transaction settings.

10. Overall Experimental Observation

The experiment demonstrates that all three JDBC approaches can insert bus-pass records correctly.

Statement is simple but requires a separate SQL execution for each record.

PreparedStatement improves query reuse and safely handles parameter values.

PreparedStatement Batch reduces the number of database communication calls by grouping insertions.

The transaction experiment also demonstrates that auto-commit can leave previously committed records in the table after a failure, whereas a single transaction can roll back the entire uncommitted import.

For bulk imports, batch insertion inside a transaction is the preferred approach among the three tested JDBC methods.

11. Final Result

The bus-pass table was created successfully in MySQL, and JDBC programs were developed using Statement, PreparedStatement, and PreparedStatement Batch.

Ten records were inserted and verified using SELECT queries and ResultSetMetaData.

Performance tests were implemented for 1,000, 10,000, and 100,000 records. The execution times were measured using System.nanoTime().

The auto-commit and transaction experiments demonstrated that previously committed records remain after a failure when auto-commit is enabled, while a single transaction can roll back all uncommitted records.

Finally, batch processing was identified as the preferred approach for nightly bulk imports, while PreparedStatement is suitable for issuing a single pass at the counter.